

Project 2: Market-Basket Analysis using IMDB

Algorithms for Massive Data

Master in Data Science for Economics
Università degli Studi di Milano

Student name: Karan Singh
Student ID: 52930A
Academic year: 2025/26
Submitted to Prof. Dario Malchiodi

Academic Integrity Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1	Dataset	3
1.1	Source	3
1.2	Subset Used	3
1.3	Fields Used	3
2	Data Organisation	3
3	Pre-processing	4
3.1	Actor Name Normalisation	4
3.2	Missing Values	4
3.3	Deduplication Within Movies	4
3.4	Basket Filtering	4
3.5	Limitations	4
4	Algorithms and Implementations	4
4.1	Market-Basket Analysis	4
4.2	Apriori: Theoretical Background	4
4.3	Custom Apriori Implementation	5
5	Scalability	5
5.1	Theoretical Complexity	5
5.2	Empirical Scalability Experiment	5
6	Experiments	6
6.1	Dataset Statistics	6
6.2	Frequent Itemsets at Minimum Support 3	6
6.3	Top Frequent Actors	6
6.4	Top Frequent Actor Pairs	7
6.5	Top Frequent Actor Triples	7
6.6	Support Threshold Experiment	7
7	Results and Discussion	7
7.1	Actor Collaboration Structure	7
7.2	Effect of Minimum Support	8
7.3	Scalability Discussion	8
7.4	Limitations and Future Directions	8
8	Conclusion	8

1. Dataset

1.1 Source

The project uses the IMDB Dataset of Top 1000 Movies and TV Shows, available on Kaggle with identifier `harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows`. The dataset is stored in a CSV file named `imdb_top_1000.csv`. Each record corresponds to one movie or TV-show entry and includes metadata such as title, genre, rating, director, overview, vote count, gross revenue, and four listed actors.

The project instructions for Project 2 define the market-basket task as follows: each movie is a basket, and the actors under `Star1`, `Star2`, `Star3`, and `Star4` are the items. This report follows exactly that structure.

1.2 Subset Used

The uploaded dataset contains 1,000 rows and 16 columns. The complete dataset is used in the main experiment. A global configuration flag is included in the notebook:

```
USE_FULL_DATASET = True
SAMPLE_SIZE = 500
```

When `USE_FULL_DATASET` is set to `False`, only the first `SAMPLE_SIZE` rows are used. The same code path is used for the full dataset and for subsamples.

1.3 Fields Used

Only the fields required for the market-basket task are used:

- `Series_Title`: used only to identify the movie corresponding to a basket.
- `Star1`, `Star2`, `Star3`, `Star4`: used as actor items inside each movie basket.

Other columns such as rating, director, genre and gross revenue are not used, because the project objective is frequent itemset mining over actor baskets rather than prediction or ranking.

2. Data Organisation

The core data structure is a list of baskets. Each basket is represented as a set of actors appearing in one movie. Formally, for a movie m_i , the basket is:

$$B_i = \{\text{Star1}_i, \text{Star2}_i, \text{Star3}_i, \text{Star4}_i\}.$$

The complete transaction database is therefore:

$$D = \{B_1, B_2, \dots, B_n\},$$

where $n = 1000$ for the full dataset.

The use of sets removes duplicate actors inside the same movie, if such a case appears. This is appropriate because market-basket analysis records whether an item appears in a transaction, not how many times the item is repeated within the same transaction.

After preprocessing, the dataset contains 1,000 baskets and 2,709 unique actors. The average basket size is approximately 3.996 actors, with basket sizes ranging from 3 to 4.

3. Pre-processing

3.1 Actor Name Normalisation

Actor names are extracted from `Star1`, `Star2`, `Star3`, and `Star4`. Leading and trailing spaces are removed using a simple string-strip operation. Empty strings and missing values are ignored.

3.2 Missing Values

There are no missing values in the four actor columns used in this project. Missing values exist in other columns, such as `Certificate`, `Meta_score`, and `Gross`, but those fields are not used for the market-basket analysis.

3.3 Deduplication Within Movies

For each movie, actor names are converted into a set. This prevents duplicated entries inside a single movie from creating artificial repeated items or invalid self-combinations.

3.4 Basket Filtering

Only non-empty baskets are kept. Since the actor columns are complete in this dataset, all 1,000 rows remain after preprocessing.

3.5 Limitations

The preprocessing has two main limitations:

- No actor-name disambiguation is performed. For example, two actors with identical names would be treated as the same item.
- Only four actors per movie are available. Therefore, larger itemsets are naturally limited, and frequent triples or quadruples are expected to be rare.

4. Algorithms and Implementations

4.1 Market-Basket Analysis

Market-basket analysis identifies groups of items that frequently occur together in a collection of transactions. In this project, movies are transactions and actors are items. The support count of an itemset X is:

$$\text{support_count}(X) = |\{B_i \in D : X \subseteq B_i\}|.$$

An itemset is frequent if its support count is at least the selected minimum support threshold.

4.2 Apriori: Theoretical Background

The Apriori algorithm is part of Chapter 6 of the course syllabus on frequent itemsets and market basket analysis. It uses the downward-closure property: if an itemset is frequent, then all of its subsets must also be frequent. Equivalently, if an itemset is not frequent, none of its supersets can be frequent.

This property reduces the number of candidate itemsets that must be counted. Instead of testing all possible actor groups, the algorithm first finds frequent single actors, then uses them to generate candidate pairs, then uses frequent pairs to generate candidate triples, and so on.

4.3 Custom Apriori Implementation

The notebook implements Apriori from scratch using Python dictionaries, counters and frozen sets. The main function is:

```
apriori(baskets, min_support, max_k)
```

The implementation follows these steps:

1. Count all individual actors and keep those with support at least `min_support`.
2. Generate candidate itemsets of size k by joining frequent itemsets of size $k - 1$.
3. Apply Apriori pruning: every $(k - 1)$ -subset of a candidate must be frequent.
4. Count candidates by scanning the movie baskets.
5. Keep only candidates meeting the minimum support threshold.
6. Stop when no larger frequent itemsets can be generated.

Because each movie contains at most four actors, the maximum itemset size is four.

5. Scalability

5.1 Theoretical Complexity

Let n be the number of baskets, b the average basket size, and C_k the number of candidate itemsets of size k . Counting candidates requires scanning the baskets and checking combinations of actors inside each basket. Since each movie has at most four actors, the number of generated combinations per basket is small:

$$\binom{4}{1} = 4, \quad \binom{4}{2} = 6, \quad \binom{4}{3} = 4, \quad \binom{4}{4} = 1.$$

For this dataset, the cost of scanning baskets is therefore approximately linear in the number of movies. In larger datasets with larger baskets, Apriori pruning becomes more important because it prevents the algorithm from evaluating many candidate supersets that cannot be frequent.

5.2 Empirical Scalability Experiment

A scalability experiment is performed by running the full Apriori pipeline on increasing fractions of the dataset: 25%, 50%, 75%, and 100%. Each runtime is averaged over repeated runs.

Dataset %	Baskets	Unique actors	Frequent itemsets	Mean runtime (s)
25%	250	818	47	0.0016
50%	500	1532	99	0.0055
75%	750	2165	203	0.0272
100%	1000	2709	299	0.0633

Table 1: Empirical scalability experiment using minimum support 3. Runtime values may vary by machine, but the experiment structure is reproducible.

The runtime increases with dataset size and with the number of frequent candidates retained. Since basket size is fixed and small in this dataset, the implementation scales mainly with the number of movie baskets and the number of generated candidates.

6. Experiments

6.1 Dataset Statistics

The full dataset contains 1,000 movie baskets and 2,709 unique actors. There are no missing values in the actor columns. One duplicated movie title appears in the title field, but this does not affect the basket analysis because each row is treated as a separate transaction.

6.2 Frequent Itemsets at Minimum Support 3

Using minimum support 3, the algorithm finds:

Itemset type	Number of frequent itemsets
Frequent actors	271
Frequent actor pairs	25
Frequent actor triples	3
Frequent actor quadruples	0

Table 2: Frequent itemsets obtained with minimum support 3.

6.3 Top Frequent Actors

Actor	Support count
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12
Christian Bale	11
Leonardo DiCaprio	11
Matt Damon	11
James Stewart	10
Michael Caine	9

Table 3: Top frequent single actors.

6.4 Top Frequent Actor Pairs

Actor pair	Support count
Daniel Radcliffe + Rupert Grint	6
Daniel Radcliffe + Emma Watson	5
Emma Watson + Rupert Grint	5
Joe Pesci + Robert De Niro	4
Tim Allen + Tom Hanks	4
Al Pacino + Diane Keaton	3
Christian Bale + Michael Caine	3
Al Pacino + Robert De Niro	3
Elijah Wood + Ian McKellen	3
Elijah Wood + Orlando Bloom	3

Table 4: Top frequent actor pairs.

6.5 Top Frequent Actor Triples

Actor triple	Support count
Daniel Radcliffe + Emma Watson + Rupert Grint	5
Elijah Wood + Ian McKellen + Orlando Bloom	3
Carrie Fisher + Harrison Ford + Mark Hamill	3

Table 5: Frequent actor triples with minimum support 3.

6.6 Support Threshold Experiment

Different minimum support thresholds are tested to observe how the number of frequent itemsets changes.

Minimum support	Actors	Pairs	Triples	Quadruples
2	641	121	25	4
3	271	25	3	0
4	145	5	1	0
5	79	3	1	0
10	9	0	0	0

Table 6: Number of frequent itemsets under different support thresholds.

As expected, increasing the minimum support sharply reduces the number of frequent itemsets. This confirms the pruning behaviour of frequent itemset mining: only highly repeated actor collaborations survive high support thresholds.

7. Results and Discussion

7.1 Actor Collaboration Structure

The results show that frequent actor pairs and triples are mostly associated with repeated film franchises or long-running collaborations. For example, Daniel Radcliffe, Emma Watson and Rupert Grint appear together frequently due to the Harry Potter films. Elijah Wood, Ian

McKellen and Orlando Bloom appear together through The Lord of the Rings films. Carrie Fisher, Harrison Ford and Mark Hamill appear together through Star Wars entries.

The algorithm identifies these patterns without using title, genre or franchise metadata. It relies only on co-occurrence inside actor baskets.

7.2 Effect of Minimum Support

The minimum support threshold has a strong effect on the result. With support 2, the algorithm finds many pairs, triples and even four quadruples. With support 3, only three triples remain. With support 10, only nine single actors remain and no larger collaborations survive. This is a useful finding because it shows that the chosen threshold controls the granularity of the discovered collaboration patterns.

7.3 Scalability Discussion

The notebook includes both a theoretical and empirical scalability analysis. The theoretical analysis shows that each movie produces a small number of actor combinations because the basket size is at most four. The empirical experiment confirms that runtime increases with the number of movie baskets and the number of generated candidates.

For a much larger movie dataset, the same Apriori implementation would still run without changing the logic. However, if future datasets include many more actors per movie or many more rows, the number of candidate combinations would increase. In that case, distributed frequent itemset mining or a more memory-efficient implementation would be appropriate.

7.4 Limitations and Future Directions

The main limitations are:

- Only four actors per movie are available, which limits the discovery of larger actor groups.
- The dataset contains only 1,000 highly rated IMDB entries, so the results are not representative of all cinema.
- Actor names are treated as plain strings, without external identity resolution.

Future work could use a larger IMDB dataset, include full cast lists, or compare the basic Apriori implementation with syllabus-related improvements such as PCY, Multistage, Multihash, or SON. These extensions are not included in the current project because the objective here is to implement the required Project 2 market-basket analysis clearly and reproducibly.

8. Conclusion

This project implemented market-basket analysis on the IMDB Movies Dataset. Each movie was represented as a basket of actors, and a custom Apriori algorithm was used to mine frequent actor itemsets. The results show clear recurring collaboration patterns, especially in major film franchises. The support-threshold experiment demonstrates how the number of frequent itemsets decreases as the threshold increases, and the scalability experiment shows that the implementation can be applied consistently across larger fractions of the dataset.

References

- [1] Agrawal, R., and Srikant, R. (1994). Fast algorithms for mining association rules. *Proceedings of the 20th International Conference on Very Large Data Bases*, 487–499.
- [2] Leskovec, J., Rajaraman, A., and Ullman, J. D. (2020). *Mining of Massive Datasets* (3rd ed.). Cambridge University Press. Available at <http://www.mmids.org>.
- [3] Kaggle. IMDB Dataset of Top 1000 Movies and TV Shows. Dataset identifier: `harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows`.